

AWS Cloud Security Lab Report

Lab 1: IAM Misconfiguration — Detection & Remediation

Prepared by: Om Fulsundar (may 2026)

1. Lab Overview

This lab focuses on one of the most common and dangerous misconfigurations in cloud environments — over-privileged IAM users. I set up a test IAM user called `test-developer-user`, intentionally granted it `AdministratorAccess`, and then simulated the kind of API activity that might indicate misuse or unauthorized enumeration. The goal was to walk through a complete detection and remediation cycle: from creating a vulnerable identity, to catching the activity in CloudTrail logs, to locking the user down to only what they actually need.

In the real world, this matters a lot. Developers and automation accounts are frequently over-privileged because it's easier to attach `AdministratorAccess` than to figure out the minimum required permissions. That shortcut becomes a massive attack surface. If an attacker gets hold of an access key that belongs to an admin-level user, they can enumerate the entire account, spin up resources, exfiltrate data from S3, or even create backdoor IAM users for persistent access. This lab is essentially a mini simulation of that threat scenario.

Real-world relevance: The 2019 Capital One breach was largely enabled by an over-privileged IAM role. Enforcing least privilege is one of the most impactful controls in any AWS security posture.

2. Objectives

Going into this lab, I had a clear set of things I wanted to accomplish:

- Create an IAM user with programmatic access and configure it with overly broad permissions (`AdministratorAccess`).
- Generate and configure access keys for CLI-based access to simulate a real developer or service account.
- Perform API calls from the CLI to demonstrate what an over-privileged identity can do.
- Enable AWS CloudTrail and verify that all management events are being captured and logged.
- Use CloudTrail Event History to detect and investigate the suspicious API activity.
- Remediate the issue by removing `AdministratorAccess` and applying the principle of least privilege with `AmazonS3ReadOnlyAccess`.

- Validate remediation by confirming blocked IAM actions and confirming permitted S3 read operations still work.

3. Tools and Services Used

Tool / Service	Purpose in This Lab
AWS IAM (Identity and Access Management)	Core service used to create users, assign permissions, and manage access keys.
AWS CLI (Command Line Interface)	Used to execute API calls against AWS from a local Windows machine (PowerShell).
AWS CloudTrail	Audit logging service that captured every API call made by the test user.
CloudTrail Event History	Console view used to search and investigate logged events by username and event type.
Amazon S3 (Simple Storage Service)	Used as the target resource to validate least-privilege access post-remediation.
PowerShell (Windows)	Local terminal environment used to run AWS CLI commands throughout the lab.

4. Lab Environment Setup

The entire lab ran on a live AWS account (Account ID: 712934828848) in the ap-south-1 (Mumbai) region. On the client side I used a Windows laptop running PowerShell with the AWS CLI installed and configured. There was no sandbox or emulated environment — everything was done on real AWS infrastructure, which made the exercise feel much more practical.

Before starting, I made sure the AWS CLI was installed and that I had the root/admin credentials available to create the IAM user and the CloudTrail trail. The test-developer-user was created fresh for this lab with no prior history, which kept the CloudTrail logs clean and easy to filter.

5. Step-by-Step Walkthrough

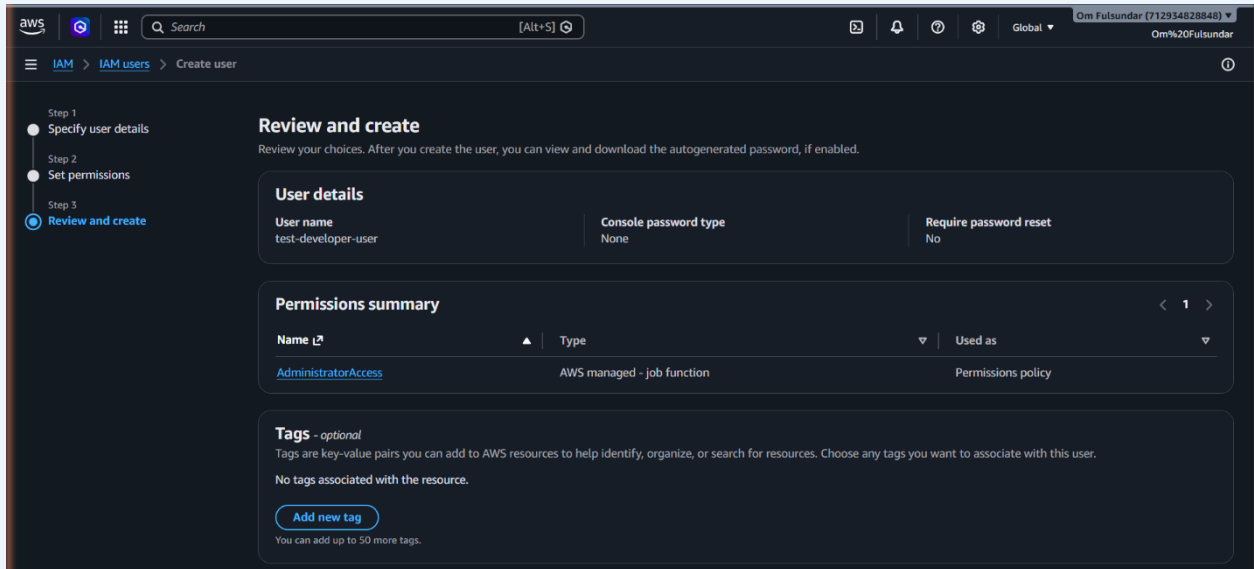
Phase 1 — IAM User Creation with Programmatic Access

I started by navigating to IAM > Users > Create user in the AWS console. I named the user test-developer-user and intentionally skipped console access (Console password type: None), since this was meant to represent a service or developer account that only uses the CLI. The most important — and most dangerous — part of this step was the permissions configuration.

On the permissions screen, I attached the AdministratorAccess policy directly to the user. This is an AWS-managed policy that grants full access to all AWS services and resources. In a real

environment, no developer account should ever have this. But for the purposes of this lab, it was exactly the misconfiguration I needed to simulate.

[SS1 — IAM User Creation]



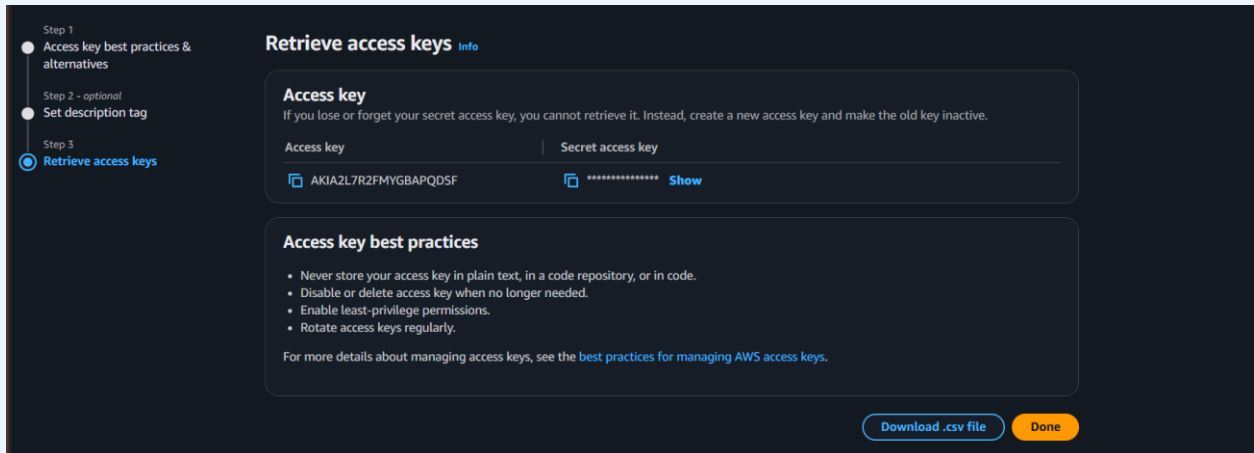
IAM console showing test-developer-user at Review and Create stage with AdministratorAccess attached

One thing I noticed here: AWS doesn't warn you or require any confirmation when you attach AdministratorAccess to a user. It just lets you do it. That's something organizations really need to control through SCPs (Service Control Policies) or permission boundaries.

Phase 2 — Access Key Generation

After creating the user, I went to the Security credentials tab and generated an access key. AWS displayed the Access Key ID and Secret Access Key one time only — you can't retrieve the secret key again after this screen. I noted both values and then configured the AWS CLI on my local machine using aws configure.

[SS2 — Access Key Generated]



AWS console Retrieve Access Keys screen showing Access Key ID: AKIA2L7R2FMYGBAPQDSF and masked secret key

The console also shows a 'Best Practices' reminder here — things like never storing keys in plaintext, rotating them regularly, and using least-privilege permissions. It's almost ironic that the warning is right there on the same screen where you can create an admin-level key with no friction. I configured the CLI profile with the new credentials and verified the connection was working before moving on.

Phase 3 — CLI Execution and Over-Privileged Access Demonstration

With the credentials configured, I ran `aws iam list-users` from PowerShell. The command succeeded immediately, returning the full user object for `test-developer-user` — including the ARN, UserID, and creation timestamp. This is a classic enumeration step that any attacker or unauthorized user would perform first to understand what accounts exist in the AWS environment.

[SS3 — CLI Execution of IAM ListUsers]

```
PS C:\Users\Lenovo> aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "test-developer-user",
      "UserId": "AIDA2L7R2FMYPIUPH6VK7",
      "Arn": "arn:aws:iam::712934828848:user/test-developer-user",
      "CreateDate": "2026-05-21T09:29:19+00:00"
    }
  ]
}
```

PS C:\Users\Lenovo> |

PowerShell output showing successful aws iam list-users response with test-developer-user details

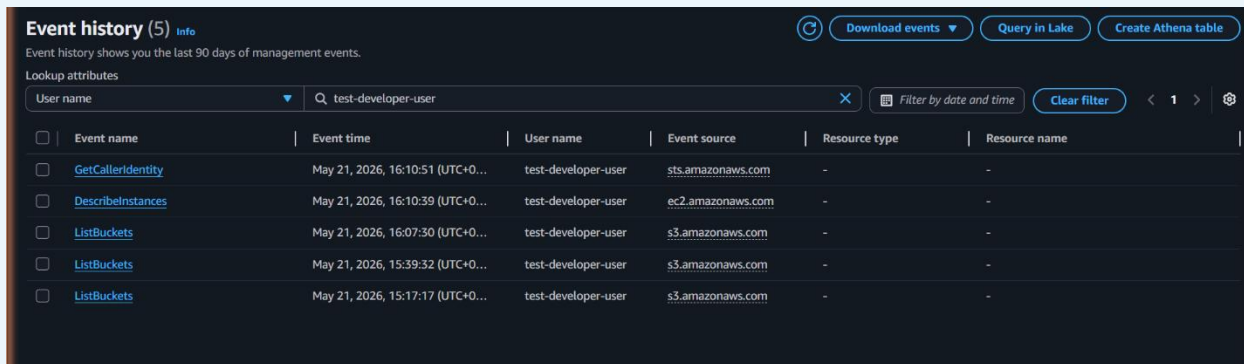
The fact that this worked confirmed the misconfiguration was live. A regular developer account should never be able to list all IAM users — that's an administrative function. But with AdministratorAccess attached, the user had zero restrictions.

Phase 4 — CloudTrail Detection and Log Review

I had already set up a CloudTrail trail (IAM_test) covering management events in ap-south-1. After running a series of API calls — including GetCallerIdentity, DescribeInstances, and ListBuckets — I went to CloudTrail > Event History and filtered by username: test-developer-user.

The results were very telling. CloudTrail had captured all five events: three ListBuckets calls, one DescribeInstances call, and one GetCallerIdentity call. Each entry showed the exact timestamp (UTC), the event source (e.g., s3.amazonaws.com, ec2.amazonaws.com, sts.amazonaws.com), and the user who triggered it. This kind of visibility is exactly what a SOC analyst needs to reconstruct a timeline of activity.

[SS4 — CloudTrail Event History]



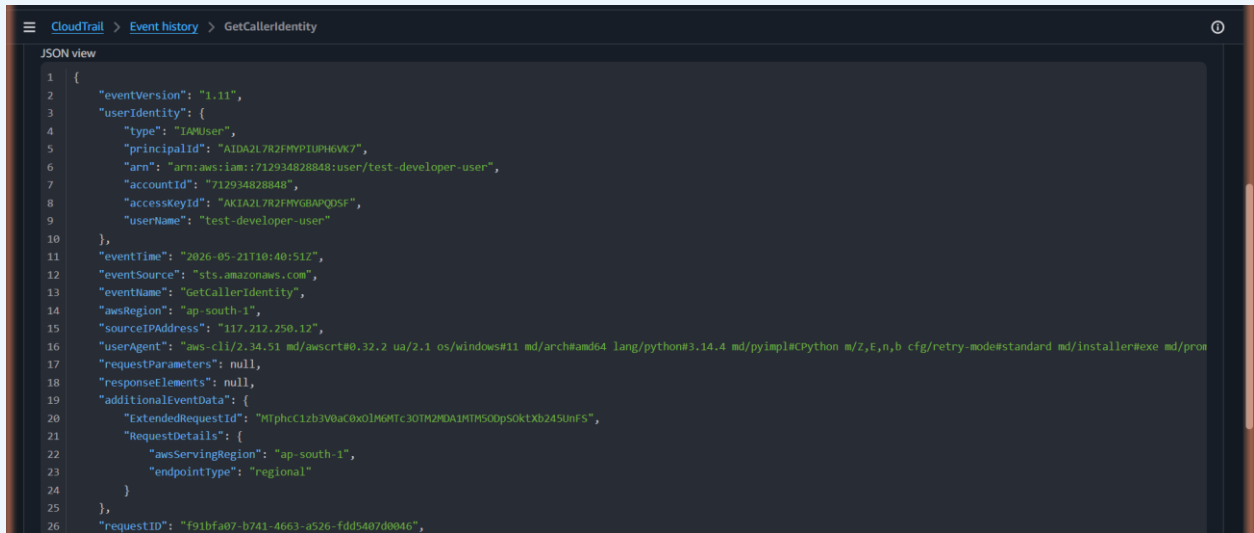
The screenshot shows the AWS CloudTrail Event History console. At the top, it says 'Event history (5)' with an 'Info' link. Below this, it states 'Event history shows you the last 90 days of management events.' There are buttons for 'Download events', 'Query in Lake', and 'Create Athena table'. A search bar for 'User name' contains 'test-developer-user'. To the right of the search bar are buttons for 'Filter by date and time' and 'Clear filter'. Below the search bar is a table with 5 events.

☐	Event name	Event time	User name	Event source	Resource type	Resource name
☐	GetCallerIdentity	May 21, 2026, 16:10:51 (UTC+0...)	test-developer-user	sts.amazonaws.com	-	-
☐	DescribeInstances	May 21, 2026, 16:10:39 (UTC+0...)	test-developer-user	ec2.amazonaws.com	-	-
☐	ListBuckets	May 21, 2026, 16:07:30 (UTC+0...)	test-developer-user	s3.amazonaws.com	-	-
☐	ListBuckets	May 21, 2026, 15:39:32 (UTC+0...)	test-developer-user	s3.amazonaws.com	-	-
☐	ListBuckets	May 21, 2026, 15:17:17 (UTC+0...)	test-developer-user	s3.amazonaws.com	-	-

CloudTrail Event History filtered by test-developer-user showing 5 events: GetCallerIdentity, DescribeInstances, and three ListBuckets calls

I then clicked into the GetCallerIdentity event to examine the full JSON log. This was probably the most instructive part of the detection phase.

[SS5 — CloudTrail Event Details (GetCallerIdentity)]



```
1 {
2   "eventVersion": "1.11",
3   "userIdentity": {
4     "type": "IAMUser",
5     "principalId": "AIDA2L7R2FMYP1U8H6VK7",
6     "arn": "arn:aws:iam::712934828848:user/test-developer-user",
7     "accountId": "712934828848",
8     "accessKeyId": "AKIA2L7R2FMYGBAPQDSF",
9     "userName": "test-developer-user"
10  },
11  "eventTime": "2026-05-21T10:40:51Z",
12  "eventSource": "sts.amazonaws.com",
13  "eventName": "GetCallerIdentity",
14  "awsRegion": "ap-south-1",
15  "sourceIPAddress": "117.212.250.12",
16  "userAgent": "aws-cli/2.34.51 md/awscrt0.32.2 ua/2.1 os/windows#11 md/arch#amd64 lang/python#3.14.4 md/pyimpl#CPython m/z,E,n,b cfg/retry-mode#standard md/installer#exe md/prom",
17  "requestParameters": null,
18  "responseElements": null,
19  "additionalEventData": {
20    "ExtendedRequestId": "MTphcC1zb3V8aC0x0LmGHTc30TM2MDA1MTM5ODp5S0ktxb245UnFS",
21    "RequestDetails": {
22      "awsServingRegion": "ap-south-1",
23      "endpointType": "regional"
24    }
25  },
26  "requestID": "f91bfa07-b741-4663-a526-fdd5407d0046",
```

Expanded JSON view of GetCallerIdentity event with userIdentity, sourceIPAddress, awsRegion, and requestParameters fields visible

The JSON log captured everything: the IAM user type, the principalId, the full ARN, the access key ID used (AKIA2L7R2FMYGBAPQDSF), the source IP address (117.212.250.12), the AWS region (ap-south-1), the userAgent string (showing this was the AWS CLI on a Windows machine), and even the request and response elements. If this were a real incident, that source IP would be the first thing I'd pivot on to determine whether the activity was legitimate or not.

6. Findings

Finding	IAM user test-developer-user was granted AdministratorAccess — full, unrestricted access to all AWS services.
Vulnerability Type	Excessive IAM Permissions / Violation of Least Privilege Principle
Severity	CRITICAL
CVSS-equivalent	High — privilege escalation and data access risk
Affected User	arn:aws:iam::712934828848:user/test-developer-user
Access Key	AKIA2L7R2FMYGBAPQDSF (Active at time of detection)
Source IP	117.212.250.12
Region	ap-south-1 (Mumbai)

Real-World Impact

If this were a real attacker who had obtained these credentials (via phishing, a leaked .env file, a compromised developer machine, or a public GitHub commit), the impact would be severe. With AdministratorAccess, they could:

- Enumerate all IAM users, roles, and policies in the account to map the identity landscape.
- List and read from all S3 buckets — potentially exfiltrating sensitive data, backups, or configuration files.
- Create new IAM users or roles with admin-level access for persistent backdoor entry.
- Terminate or modify EC2 instances, disrupting production workloads.
- Delete or disable CloudTrail trails to cover their tracks.
- Access secrets stored in AWS Secrets Manager or Parameter Store.

This is not a theoretical risk. Access key leakage is one of the most common initial access vectors in AWS breaches. Over-privileged keys make the blast radius of any credential compromise catastrophically large.

7. Detection Method

Detection was achieved through AWS CloudTrail, which was logging all management events in the ap-south-1 region. The key detection steps were:

1. CloudTrail trail (IAM_test) was active and capturing management events before the activity occurred — this is critical, because if the trail had been created after, the events would already be gone.
2. Event History was filtered by the specific username (test-developer-user), which immediately surfaced all API calls made by that identity.
3. The JSON detail view of the GetCallerIdentity event provided forensic-quality data: exact access key used, source IP, region, timestamp, and user agent string.
4. The combination of IAM enumeration calls (list-users) and resource enumeration calls (DescribeInstances, ListBuckets) in a short time window would be a strong indicator of reconnaissance activity in a real environment.

In a production SOC environment, this kind of detection would ideally be automated. AWS GuardDuty, for example, has built-in detections for IAM credential misuse, unusual API call patterns, and access from anomalous IPs. A SIEM rule could also alert on a burst of diverse API calls from a single IAM user within a short window.

8. Remediation

Step 1 — Remove AdministratorAccess

In the IAM console, I navigated to the user's Permissions tab and removed the AdministratorAccess policy. This immediately cut off all permissions the user previously had. At this point the account was effectively locked out of everything.

Step 2 — Attach Least-Privilege Policy

I then attached the AWS-managed AmazonS3ReadOnlyAccess policy. This policy grants only s3:Get* and s3:List* actions — nothing else. It's a reasonable minimum for a developer who needs to read from S3 but shouldn't be touching IAM, EC2, or any other service.

[SS6 — IAM Permissions Remediation]

test-developer-user

Info

Delete

Summary

ARN
arn:aws:iam::712934828848:user/test-developer-user

Created
May 21, 2026, 14:59 (UTC+05:30)

Console access
Disabled

Last console sign-in
-

Access key 1
AKIA2L7R2FMYGBAPQDSF - Active
Used today. Created today.

Access key 2
Create access key

Permissions

Groups

Tags

Security credentials

Last Accessed

Permissions policies (1)

Remove

Add permissions

Permissions are defined by policies attached to the user directly or through groups.

Search

Filter by Type
All types

< 1 >

☐

Policy name

▲

Type

▼

Attached via

☐

AmazonS3ReadOnlyAccess

AWS managed

Directly

IAM user page showing AdministratorAccess removed and AmazonS3ReadOnlyAccess attached as the only policy

Before vs. After Comparison

Before Remediation	After Remediation
AdministratorAccess (full AWS access)	AmazonS3ReadOnlyAccess (S3 list/get only)
aws iam list-users → SUCCESS	aws iam list-users → ACCESS DENIED
aws s3api list-buckets → SUCCESS	aws s3api list-buckets → SUCCESS (still permitted)

Could create/delete IAM users	Cannot perform any IAM actions
Could terminate EC2 instances	Cannot perform any EC2 actions
Could read/write/delete S3 objects	Can only list and read S3 buckets and objects

Validation — Access Denied

After applying the new policy, I re-ran `aws iam list-users --region ap-south-1` from PowerShell. This time the response was an `AccessDenied` error: the user was explicitly told it was not authorized to perform `iam:ListUsers` because no identity-based policy allows that action. That's exactly what we want to see.

[SS7 — Remediation Validation (Access Denied)]

```
PS C:\Users\Lenovo> aws iam list-users --region ap-south-1

aws: [ERROR]: An error occurred (AccessDenied) when calling the ListUsers
operation: User: arn:aws:iam::712934828848:user/test-developer-user is not
authorized to perform: iam:ListUsers on resource: arn:aws:iam::7129348288
48:user/ because no identity-based policy allows the iam:ListUsers action
PS C:\Users\Lenovo> |
```

PowerShell showing AccessDenied error for `aws iam list-users --region ap-south-1` after policy change

Validation — Permitted S3 Action Still Works

I also confirmed that the user's legitimate use case — reading from S3 — was still functional. Running `aws s3api list-buckets --region ap-south-1` returned two S3 buckets (both CloudTrail log buckets), confirming the `AmazonS3ReadOnlyAccess` policy was working correctly.

[SS8 — Post-Remediation S3 Verification]

```
PS C:\Users\Lenovo> aws s3api list-buckets --region ap-south-1
{
  "Buckets": [
    {
      "Name": "aws-cloudtrail-logs-712934828848-70f5251b",
      "CreationDate": "2026-05-21T10:00:23+00:00",
      "BucketArn": "arn:aws:s3:::aws-cloudtrail-logs-712934828848-70f5251b"
    },
    {
      "Name": "aws-cloudtrail-logs-712934828848-a15b60b8",
      "CreationDate": "2026-05-21T10:25:42+00:00",
      "BucketArn": "arn:aws:s3:::aws-cloudtrail-logs-712934828848-a15b60b8"
    }
  ],
  "Owner": {
    "ID": "3da45a15b80604dabf97dd60aebfb70bafb824fd7e633f8cb61ad19fa64173d0"
  },
  "Prefix": null
}
```

PowerShell output showing successful aws s3api list-buckets returning two CloudTrail buckets

This validation step is important: remediation shouldn't just block the bad — it should confirm the good still works. Confirming the allowed action proves the policy is correctly scoped, not just maximally restrictive.

9. Key Learnings

- Least privilege is not just a best practice — it's a critical control. The difference between AdministratorAccess and AmazonS3ReadOnlyAccess is the difference between a catastrophic breach and a contained one. Keeping permissions as narrow as possible limits the blast radius of any credential compromise.
- CloudTrail needs to be set up before an incident, not after. One of the biggest takeaways for me was how dependent detection is on having logging already in place. If I hadn't created the CloudTrail trail before running the API calls, there would have been nothing to investigate. In a real cloud environment, trails should be enabled from day one with log integrity validation turned on.
- The GetCallerIdentity event is a goldmine for forensics. This API call captures the exact identity, source IP, access key, and user agent. It's often one of the first things an attacker or a penetration tester runs to verify their access — and fortunately, it's also one of the most informative events you'll find in CloudTrail logs during an investigation.
- What surprised me most: AWS has no native guardrails that prevent you from attaching AdministratorAccess to a user — it's completely silent. In a real organization, this should

be controlled through IAM permission boundaries or SCPs at the Organizations level so that even account admins can't grant more permissions than a defined maximum.

- Remediation must be validated, not assumed. Simply removing a policy isn't enough — you need to confirm both that the over-privileged action is now blocked and that the permitted actions still function. This prevents accidental service disruption while ensuring the security fix actually works.

10. Conclusion

This lab walked through a complete attack lifecycle in miniature — from misconfiguration to detection to remediation and validation. Starting with an over-privileged IAM user, I was able to demonstrate how easy it is for a credential with AdministratorAccess to enumerate an AWS environment, and more importantly, how CloudTrail makes that activity visible and attributable.

The remediation cycle here — removing AdministratorAccess and replacing it with AmazonS3ReadOnlyAccess — is something that security engineers and cloud teams do in real environments when they're trying to clean up identity sprawl or respond to a finding from AWS IAM Access Analyzer or a security audit. Knowing how to do it correctly, and knowing how to validate it, is a practical skill that translates directly to cloud security engineering and SOC work.

For anyone working in or moving toward a SOC or cloud security role, this exercise is a good foundation. The three pillars it covers — identity misconfiguration, log-based detection, and least-privilege remediation — come up constantly in cloud security incidents. CloudTrail is the backbone of AWS forensics, and understanding how to read its event logs, filter by identity, and correlate API calls into a timeline is essential for any incident responder working in an AWS environment.

Key takeaway: In cloud security, identity is the new perimeter. An over-privileged IAM identity is just as dangerous as an open firewall rule — and often harder to notice until something goes wrong.